SYS_TOP Verification Report

Test cases for the four supported UART commands

1. Test environment

Item	Value
DUT	SYS_TOP (UART RX/TX, SYS_CTRL, RegFile, ALU, Clock Gating, ASYNC FIFO, Data/Reset Synchronizers, Clock Dividers)
Testbench	tb_SYS_TOP (Top/system_tb.v) - drives UART_RX_IN, samples UART_TX_O, self-checking
Clocks	REF_CLK = 50 MHz, UART_CLK = 3.6864 MHz
UART configuration	REG2 = 0x81: parity enabled, even parity, prescale 32; REG3 = 0x20: divide ratio 32 (115200 baud)
Frame format	1 start bit, 8 data bits (LSB first), even parity bit, 1 stop bit
Simulator	Icarus Verilog 12.0. Times below are simulation time in ns.
Overall result	21 checks executed, 21 passed, 0 failed

2. Supported commands

Command	Opcode	Frames (in order)	Response	Test IDs
Register File Write	0xAA	3: AA, DATA, ADDRESS	None	RF_WR_01 - RF_WR_06
Register File Read	0xBB	2: BB, ADDRESS	1 frame: register data	RF_RD_01 - RF_RD_04
ALU Operation with operands	0xCC	4: CC, OPERAND A, OPERAND B, ALU FUN	1 frame: ALU result (low byte)	ALU_OP_01 - ALU_OP_14
ALU Operation without operands	0xDD	2: DD, ALU FUN (uses REG0 / REG1)	1 frame: ALU result (low byte)	ALU_NOP_01

3. Results summary

Command	Test cases	Passed	Failed
Register File Write	6	6	0
Register File Read	4	4	0
ALU Operation with operands	14	14	0
ALU Operation without operands	1	1	0
Error handling (parity / framing)	2	2	0
Total	**27**	**27**	**0**

Register File Write has no response frame, so each write is checked indirectly: by a later read-back, by a later ALU result, or by the UART continuing to work with the new configuration. The testbench scoreboard counts 21 direct checks (2 reads + 14 ALU + 1 no-operand ALU + 2 error flags + 2 recovery reads). The six write cases are verified through those checks.

4. Command 1 - Register File Write (0xAA)

ID	Description	Frames sent	Data	Addr	Verified by	Result
RF_WR_01	Configure UART: parity enable, even, prescale 32	AA 81 02	0x81	0x02	All later traffic decodes correctly at the configured format (implicit)	PASS
RF_WR_02	Configure clock divider ratio = 32	AA 20 03	0x20	0x03	All later traffic runs at 115200 baud (implicit)	PASS
RF_WR_03	Write normal register 0x04	AA 55 04	0x55	0x04	Read-back RF_RD_01 returns 0x55	PASS
RF_WR_04	Write normal register 0x05	AA A5 05	0xA5	0x05	Read-back RF_RD_02 returns 0xA5	PASS
RF_WR_05	Write ALU operand A (REG0)	AA 0C 00	0x0C	0x00	ALU_NOP_01 uses REG0 = 0x0C	PASS
RF_WR_06	Write ALU operand B (REG1)	AA 03 01	0x03	0x01	ALU_NOP_01 uses REG1 = 0x03	PASS

5. Command 2 - Register File Read (0xBB)

ID	Description	Frames sent	Expected	Received	Sim time (ns)	Result
RF_RD_01	Read back register 0x04	BB 04	0x55	0x55	1367759	PASS
RF_RD_02	Read back register 0x05	BB 05	0xA5	0xA5	2055375	PASS
RF_RD_03	Read register 0x04 after a parity-error frame (recovery)	BB 04	0x55	0x55	11603663	PASS
RF_RD_04	Read register 0x05 after a framing-error frame (recovery)	BB 05	0xA5	0xA5	12082383	PASS

6. Command 3 - ALU Operation with operands (0xCC)

ID	Operation	Computation (hex)	Frames sent	Expected	Received	Sim time (ns)	Result
ALU_OP_01	ADD	0A + 05	CC 0A 05 00	0x0F	0x0F	2629839	PASS
ALU_OP_02	SUB	0A - 05	CC 0A 05 01	0x05	0x05	3204303	PASS
ALU_OP_03	MUL	05 x 04	CC 05 04 02	0x14	0x14	3778767	PASS
ALU_OP_04	DIV	14 / 04	CC 14 04 03	0x05	0x05	4353231	PASS
ALU_OP_05	AND	F0 & 0F	CC F0 0F 04	0x00	0x00	4927695	PASS
ALU_OP_06	OR	F0 | 0F	CC F0 0F 05	0xFF	0xFF	5502159	PASS
ALU_OP_07	NAND	~(F0 & 0F)	CC F0 0F 06	0xFF	0xFF	6076623	PASS
ALU_OP_08	NOR	~(F0 | 0F)	CC F0 0F 07	0x00	0x00	6651087	PASS
ALU_OP_09	XOR	F0 ^ 0F	CC F0 0F 08	0xFF	0xFF	7225551	PASS
ALU_OP_10	XNOR	~(F0 ^ 0F)	CC F0 0F 09	0x00	0x00	7800015	PASS
ALU_OP_11	CMP A = B	55 == 55	CC 55 55 0A	0x01	0x01	8374479	PASS
ALU_OP_12	CMP A > B	55 > 20	CC 55 20 0B	0x01	0x01	8948943	PASS
ALU_OP_13	SHIFT A >> 1	80 >> 1	CC 80 00 0C	0x40	0x40	9523407	PASS
ALU_OP_14	SHIFT A << 1	04 << 1	CC 04 00 0D	0x08	0x08	10097871	PASS

7. Command 4 - ALU Operation without operands (0xDD)

ID	Description	Precondition	Frames sent	Expected	Received	Sim time (ns)	Result
ALU_NOP_01	ADD using stored operands	REG0 = 0x0C, REG1 = 0x03 (RF_WR_05, RF_WR_06)	DD 00	0x0F	0x0F	11124943	PASS

8. Error handling (additional tests)

Each test sends a corrupted copy of a valid command byte (0xBB). The frame must raise the error flag and be dropped. The recovery reads RF_RD_03 and RF_RD_04 show that the dropped frame did not leave the command FSM waiting for an address byte.

ID	Description	Stimulus	Expected	Observed	Result
ERR_01	Parity error	Frame BB with inverted parity bit	parity_error asserts; frame dropped	parity_error asserted at 11216600 ns; RF_RD_03 passes	PASS
ERR_02	Framing error	Frame BB with stop bit = 0	framing_error asserts; frame dropped	framing_error asserted at 11704024 ns; RF_RD_04 passes	PASS

9. Notes

• The A > B compare returns 0x01 in the ALU and the testbench expects 0x01. The specification does not define the compare result value.

• ALU function codes follow the specification: 0xC = shift right, 0xD = shift left. There is no A < B operation.

• The testbench contains one test for the no-operand ALU command. More functions can be added to ALU_NOP_xx using the same task.

• **Bugs fixed before this run:** TX used live FIFO data (parity and data corruption); ALU codes for shifts were off by one; ALU valid stayed high because the gated clock stopped too early; RX raised false framing errors after every frame.